CMPE-679 Lab 3

TA: Ishan Renjen

March 17, 2026

Lab 3 repo/paper
Lab 3 Dataset Option
Other Dataset options and examples
useful info - better way of upsampling with convolutions
explanation of WGAN-GP
DCGAN pytorch tutorial

Due April 14th

Introduction

This lab is implementing WaveGAN, one of the earliest papers in Audio ML, espeically with generating synthetic audio. The goal of the lab will be to create an implementation of WaveGAN which can generate audio samples of about 1 second. The model can be trained on a variety of datasets, some of which linked are: drum sounds, spoken words, bird sounds, piano music. Any training data is valid for this lab, including those not linked. The lab will consist of implementing a PyTorch model given the paper, Tensorflow implementation, and dataset options.

Model

The core structure of the model is very similar to DCGAN, with a pytorch tutorial linked above. The generator and discriminator are just repeated convolutions doing upsampling/downsampling. Even though DCGAN uses images, the structure and general principle of the model is the same since it still relies on convolutions. The main challenges with the model implementation are: Phase Shuffle, Gradient Penalty, and properly preprocessing the input data.

When upsampling data with convolutions, this can easily lead to artifacts present in the data. There are a few ways of upsampling, but regardless of the method it still relies on convolutions, which are repeated strides across a vector/matrix. These artifacts are regular in the generated data since convolutions are repeated consistently over an image (this is explained in detail in the link provided above). When using this sort of method in a GAN, a discriminator can easily find these patterns and differentiate real vs. fake data. To attempt to resolve this, WaveGAN introduces a random phase alteration to the discriminator to make it harder to notice these artifacts when comparing real and generated audio.

One problem with GAN's is that training is unstable, with problems like mode collapse or vanishing/exploding gradients. One solution to the gradient issue is using a gradient penalty, which adds a regularization term that encourages the critic (basically the discriminator) to have a gradient output close to 1. This way, the gradient will not drop to 0 or explode. The difference with WaveGAN's discriminator and a typical discriminator is that it uses a WGAN-GP training strategy, where the discriminator doesn't output real vs. fake necessarily, but computes the Wasserstein Distance between the real and generated distributions. This is essentially replacing a standard loss function. The advantage of this approach is that it is more stable for GAN training due to controlling gradients and

evaluating the distance between real and generated distributions instead of MSE over the entire output, for example.

One thing to consider when preprocessing the data is that the sample rate is very important to standardize. In a dataset of images, it is important to maintain the same size, or the same number of layers. For audio, these constraints change. The factors to keep in mind are: single-channel (ideally), aim for 16 KHz sample rate and 16384 slices to get it to one second audio samples (this is explained through the paper and tensorflow code). You may need to change the sample rate of the data or pad the data since everything should be at the specified sample rate and slice len. Torchaudio does all of that for you. You will also need to implement a train/test split if one is not provided in the chosen dataset.

One important note is that GAN training is hard, and the results are not as easy to quantify as other models. The main metric for this lab will be the quality of the generated audio. Save samples at regular iterations (every epoch) to see how training progress is going. Implementing Inception Score will be bonus points.

1 Steps to get started

1. Read through WaveGAN paper.
2. Read through WaveGAN code, understand the mechanics of the model
3. Read through the provided links to understand how DCGAN works, how to upsample with convolutions, and how WGAN-GP works
4. Write the data preprocessing code, including train/test split if needed.
5. Implement a basic working model without Phase Shuffle in PyTorch - basic working model requires gradient penalty.
6. Refine the model, implement Phase Shuffle.

Deliverables

Google Drive link to a folder with:

- Jupyter Notebook (.ipynb) with all the code for the assignment - The code should be fully functional (run a full epoch of training/eval or no points will be given).
- Model file (.pth) that was used to generate audio samples.
- Short informal report with lab description, results, issues encountered, training loss, testing loss, validation loss, translations, and potential improvements - The report must be included or no points will be given.
- Jupyter Notebook/Python file used exlusively for generating data - load in the .pth and run it to generate new audio clips from random vectors.
- 5 different generated audio samples - include 5 distinct audio clips generated from the model. These clips must be of what the model was trained on - not noise.

2 Grading

- **Functional Jupyter Notebook and Model:** 50 pts
- **5 distinct audio clips(not noise):** 30 pts
- **Report:** 20 points

  - Summary of lab (explain purpose of lab, data, model, training method and specific details of model implementation): 10 points
  - Training, Testing loss over all epochs trained: 5 points
  - Conclusion (explain issues, potential improvements, etc): 5 points

- **Bonus:**
  - 5 points: Feedback on lab - anything you would like to see changed.
  - 10 points: Compute an Inception Score for the audio clips and explain what the score is and what your result means.
  - 20 points: Find/create your own dataset, modify the model, and use it to generate audio clips of 4+ seconds.